

Sentinel: Technical Documentation

Deterministic AutoML and MLOps for predictive maintenance. Built by John Tewolde for the Prototype Phase of the ABB Accelerator (Theme 1).

1. Problem and approach

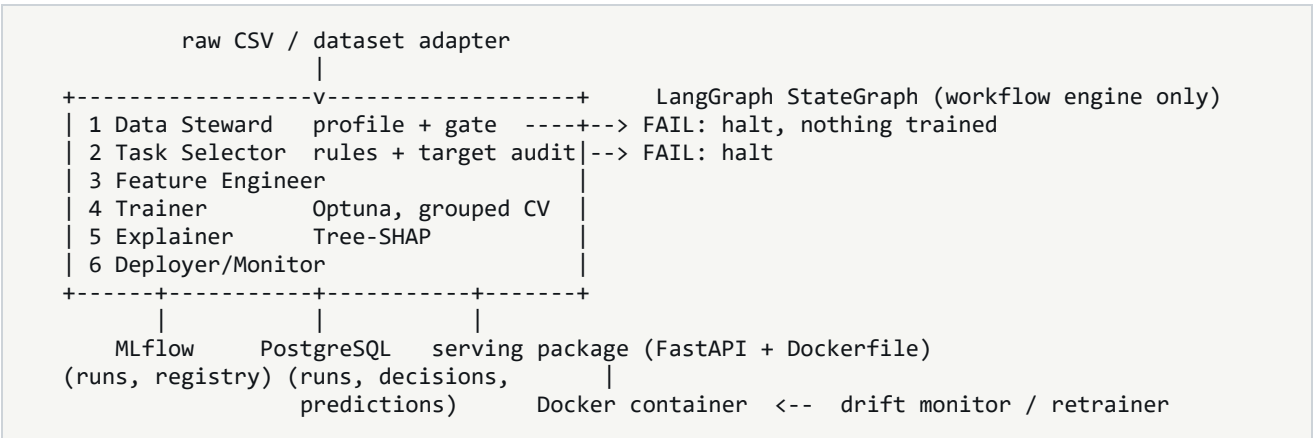
Predictive maintenance tools tend to fail at one point. The model flags a machine, or misses one, and nobody can say why, so nobody acts on it. Putting a generative model in charge of the workflow makes that worse, because its reasoning can't be reproduced or audited.

Sentinel takes a plant's raw sensor data to a deployed, explainable failure-prediction model, and it holds to one rule the whole way: every decision is a fixed rule, a statistical test, or a cross-validated metric. No language model is called anywhere. Machine learning does the one job it's good at, which is predicting failures.

That rule has three practical effects:

- **Reproducible.** The same data, thresholds and seed give the same decision log, the same leaderboard and the same run hash.
- **Auditable.** Each decision records the rule that fired, the numbers it read, and the threshold it compared them against.
- **Explainable.** Explanations are Tree-SHAP values turned into sentences by fixed templates, so there's no free-form text to second-guess.

2. Architecture



Component	File	Responsibility
Config	sentinel/config.py	Every threshold and the seed, in one place. Part of the run hash.

Component	File	Responsibility
Structure detection	<code>sentinel/structure.py</code>	Finds the entity column and the time counter from the data.
Data Steward	<code>sentinel/steward.py</code>	Scorecard and quality gate, in two stages.
Selector	<code>sentinel/selector.py</code>	Task framing, target derivation, ranking metric, holdout split.
Features	<code>sentinel/features.py</code>	Serializable feature spec and transforms, shared with the serving container.
Trainer	<code>sentinel/trainer.py</code>	Seeded Optuna search and the cross-validated leaderboard.
Explainer	<code>sentinel/explainer.py</code>	SHAP contributions, ranked drivers, sentence templates, reliability table.
Deployer	<code>sentinel/deployer.py</code>	Serving package, in-process parity test, container smoke test.
Monitor and lifecycle	<code>sentinel/monitor.py</code> , <code>sentinel/lifecycle.py</code>	PSI and KS drift, the retrain rule, champion/challenger.
Registry and store	<code>sentinel/registry.py</code> , <code>sentinel/store.py</code>	MLflow tracking and registry; SQLAlchemy persistence.
Graph	<code>sentinel/graph.py</code>	The six nodes and the conditional gate edges.
API and UI	<code>api/main.py</code> , <code>web/</code>	REST, server-sent events, and a single-page UI.

2.1 How the stations line up with ISO 13374

ISO 13374 describes condition monitoring as six functional blocks. I arranged the stations to follow them, which helped me decide where each piece of logic belongs. It's an organizing analogy; Sentinel hasn't been assessed for conformance to the standard.

ISO 13374 block	Function	Where it lives in Sentinel
1. Data Acquisition	Get the data in	Dataset adapters and the CSV loader (<code>sentinel/data.py</code>). There's no live streaming yet.
2. Data Manipulation	Clean and shape it	Data Steward (<code>sentinel/steward.py</code>) and Feature Engineer (<code>sentinel/features.py</code>)
3. State Detection	Compare against a healthy baseline	The early-life baseline and the z-score of each driver
4. Health Assessment	Judge current condition	The grouped cross-validation leaderboard and the Tree-SHAP explainer (<code>sentinel/trainer.py</code> , <code>sentinel/explainer.py</code>)
5. Prognostic Assessment	Look ahead	The alarm horizon (30 cycles, which is 0.15 times the median life) and the measured hit rates (<code>sentinel/selector.py</code>)
6. Advisory Generation	Tell someone what to do	Alert sentences, the risk-ranked fleet list, and a printable run report (<code>api/</code> , <code>web/</code>)

3. The decision rules

Each row below is a rule the pipeline applies, with its threshold from `config.py`. Every time one fires, it's written to the decision ledger along with its inputs.

Rule	Reads	Threshold	Outcome
Sample size	row count	warn < 1000, fail < 100	a fail halts the run
Missing values	worst per-column missing fraction	warn > 5%, fail > 30%	a fail halts the run
Duplicates	share of duplicate rows	warn > 1%	reported
Constant columns	unique values per column	1	column dropped
Identifier columns	one unique value per row, or more than 50 categories	n/a	column dropped
Multicollinearity	pairwise r, VIF	$r \geq 0.95$, $VIF > 10$	reported (trees tolerate it)
Distribution shift	two-sample KS, early vs late life (or first vs second half)	$D \geq 0.2$, $p < 0.01$	reported; expected for degradation data
Structure	contiguous entity blocks with a +1 time counter	99% of steps	run-to-failure framing
Alarm horizon	median life	0.15 x median life	target = fails within the horizon
Target type	distinct target values	binary, or more than 10 continuous	classification or regression
Class balance	minority share	warn < 10%, fail < 0.5%	a fail halts the run
Ranking metric	minority share	< 10%	average precision; otherwise ROC-AUC (RMSE for regression)
Class weighting	minority share	< 30%	balanced weights on every candidate
Leakage guard	$P(\text{target} = 1 \text{ given flag} = 1)$ for each 0/1 column	$\geq 98\%$, support ≥ 10	column dropped
Champion	cross-validated metric	rank 1	selected; the holdout is never consulted
Alert threshold	out-of-fold scores	maximizes F1	fixed and stored with the model
Drift	PSI of the top-10 important features, early-life window	watch 0.10, retrain 0.25	retrain rule
Promotion	challenger vs incumbent on unseen units	strictly better	promote or keep

4. Reproducibility

Getting the same answer twice took more care than I expected, so here's everything that's pinned down.

- **Seeds.** `SEED = 42` covers Optuna's TPE sampler, the LightGBM, XGBoost and scikit-learn estimators, fold splitting and holdout splitting.
- **Fixed parallelism.** `N_JOBS = 4` is a constant instead of `os.cpu_count()`, and LightGBM runs with `deterministic=True, force_row_wise=True`.
- **Stable ordering.** Series are stable-sorted by (entity, time), and leaderboard ties break alphabetically.
- **The run hash** is the SHA-256 of a canonical JSON made from a fingerprint of the input frame, all options and thresholds, the full decision log, and the leaderboard (family, parameters, CV scores). Wall-clock times are left out on purpose, and floats are rounded to six places.
- **Pinned libraries.** The hash depends on library versions, so `requirements.txt` pins exact versions and the Docker image installs from it.
- **Tested.** `test_pipeline_is_deterministic` runs the whole pipeline twice in separate directories and asserts identical hashes, leaderboards and decision logs. I've also run the full pipeline on NASA C-MAPSS from scratch several times, in separate processes and in the Docker Compose stack, and every run produced the hash `f206c5c62729d1464cce14aa15775ea3e5593d5ec7529184c1411ce9ffc01c2c`.

5. Feature engineering

For time-series data, each non-constant sensor becomes five features per time step, computed per unit over trailing windows: the current value, a 5-step mean, a 5-step standard deviation, a 10-step least-squares slope, and a 5-step change. The start of a series is padded with its first value, so training and serving agree. The time counter stays in as a feature too. On C-MAPSS FD001 that works out to 17 sensors $\times 5 + 1 = 86$ features. For tabular data, numeric columns pass through and categoricals are one-hot encoded with a category list frozen at training time.

Several model libraries break on brackets, quotes and commas in feature names, so names are sanitized for training and mapped back to the original column names for display.

6. Training and evaluation

The candidates are LightGBM, XGBoost, Random Forest and a linear baseline (logistic or ridge). Each gets a seeded Optuna TPE search (6/6/3/3 trials in the default "fast" budget, 20/20/8/6 in "full") scored by 5-fold cross-validation grouped by unit, so no unit ever sits in both the training and validation folds. The primary metric is ROC-AUC, or average precision when failures are rare, or RMSE for regression. The alert threshold is the out-of-fold score that maximizes F1.

After ranking, every family is refit on all the training data and scored on the holdout. That's for transparency only; the holdout never influences which model wins.

7. Explainability

Contributions come from Tree-SHAP. For LightGBM and XGBoost I use the library's native `pred_contrib`, which is the same algorithm and doesn't need the `shap` package in the serving image. `parity_check`

confirms it matches `shap.TreeExplainer` on 400 sampled rows, and the maximum absolute difference on the C-MAPSS run is 0.0.

For a given prediction, the top drivers are ranked by absolute contribution. Each one carries a z-score against the fleet baseline and its share of the explained signal, and the sentence is assembled from a fixed template:

Engine 76 ALERT at cycle 205: failure risk >99%. Ps30 static pressure at HPC outlet, 5-cycle average above baseline (+1.9 sd), 15% of the signal; ...

Confidence is measured from the data. It's the out-of-fold hit rate of the score band the prediction falls in: historically, 94% of scores in that band were followed by failure within the alarm horizon (n=2635).

8. Deployment

`deployer.package` writes a self-contained folder for each model version:

```
model.(txt|json|joblib)  feature_spec.json  reference.json  metadata.json
sentinel/features.py      sentinel/explainer.py  serve.py
Dockerfile  requirements.txt  README.md
```

The service exposes `GET /health`, `GET /metadata` and `POST /predict`, which takes raw sensor rows and returns risk, alert level, confidence and the ranked drivers. `features.py` and `explainer.py` are copied verbatim from training, so training and serving can't disagree about how a feature is computed.

Two automated checks cover this. The first runs on every pipeline run and the second runs on demand:

1. **Parity test.** The generated service is imported the way the container would import it, and its scores are compared with the trained model's (`SERVING_PARITY`, maximum difference 0.0).
2. **Container smoke test.** It builds the image, starts it, calls `/health` and `/predict`, and compares the scores. It needs the Docker socket, which `docker compose` mounts by default.

Models are logged to MLflow with their parameters, metrics, decision log, scorecard, leaderboard and feature spec, then registered as `sentinel-<dataset>` with the alias `champion`.

9. Monitoring and retraining

The monitor stores quantile-bin reference distributions for every feature. Live data is compared with the training distribution using the Population Stability Index and a KS test. A degrading machine drifts on its own as it wears, so comparing whole lives would fire constantly; the comparison uses the healthy early-life window (the first 50 time steps) instead. The retrain rule is fixed: any of the ten most important features at PSI 0.25 or above.

When the rule trips, `lifecycle.retrain` refits the champion's model family with the same parameters on the training data plus half of the drifted units. It then scores the incumbent and the challenger on the other half. The challenger is promoted, registered as the next MLflow model version and packaged as a new service folder only if it's strictly better.

The UI's drift scenario adds a calibration offset (a number of training standard deviations) to one sensor. That offset is simulated; the detection, retraining and promotion that follow are real.

10. HTTP API

Method and path	Purpose
GET <code>/api/mode</code> , GET <code>/api/datasets</code>	Capabilities and available datasets
POST <code>/api/upload</code>	Upload a CSV
POST <code>/api/runs</code>	Start a run with <code>{dataset, budget, task, target, horizon}</code>
GET <code>/api/runs</code> , GET <code>/api/runs/{id}</code> , GET <code>/api/runs/{id}/trace</code>	List runs, fetch a run, fetch its events
GET <code>/api/runs/{id}/events</code>	Server-sent events while the run executes
POST <code>/api/runs/{id}/predict</code>	Score raw rows through the generated service; logged to PostgreSQL
POST <code>/api/runs/{id}/deploy</code>	Container build and smoke test
POST <code>/api/runs/{id}/drift</code> , POST <code>/api/runs/{id}/retrain</code>	Drift scenario and champion/challenger
GET <code>/api/runs/{id}/lineage</code>	MLflow versions

10.1 Security and offline operation

Sentinel is a prototype, and nobody has assessed it against IEC 62443 or any other security standard. A few choices do make it easier to run on an isolated plant network:

- **No external calls at run time.** Fonts, stylesheets and scripts are bundled locally (`web/fonts/` , IBM Plex under the SIL OFL), so the web app never requests anything from Google Fonts or a CDN. Once the images and data are pulled, it runs without Internet access.
- **The Docker socket is optional.** The container smoke test needs `/var/run/docker.sock` . Set `DOCKER_SOCKET=/dev/null` to leave it out; the Deploy step then reports that Docker is unavailable, skips the smoke test and still writes the service package.
- **Bounded file access.** Dataset and run paths are resolved and must stay inside the uploads, samples or runs folder, which blocks directory traversal.
- **Upload limits.** Uploads are capped at 50 MB, limited to `.csv` and `.txt` , and rejected if the file doesn't parse as CSV.

11. Setup and testing

```
python scripts/fetch_data.py          # NASA C-MAPSS FD001 and UCI AI4I 2020 into ./data
docker compose up --build             # app :8000, MLflow :5000, PostgreSQL
python -m sentinel.cli run cmapss_fd001 --budget fast    # or run headless
pytest -q                             # 19 tests on synthetic data
```

The tests cover structure detection, the steward checks and the failing gate, the leakage guard, the closed-form horizon rule, feature ordering and window correctness against pandas, PSI behavior, end-to-end determinism, champion selection, serving parity, SHAP parity, the registry and the regression path. One more is a static import audit that fails if any language-model client appears in the package.

12. Scalability and feasibility

- **Data.** Feature computation is vectorized per unit. Training time scales with rows x features x trials, and the trial budget is a parameter. The same code path has run on C-MAPSS (20,631 rows, 86 features) and on AI4I (8,000 training rows).
- **Storage.** PostgreSQL holds runs, decisions and predictions, and an MLflow server with a proxied artifact store holds the models. Both are standard services that scale horizontally.
- **Serving.** Each model version is an independent, stateless container with one small dependency set, so it can sit behind any load balancer and roll back by tag.
- **Adoption.** Nothing leaves the machine and there are no external API calls at run time, which matters on plant networks. A reviewer can also sign off on the decision ledger and run hash, because both are concrete records.

13. Limitations and roadmap

- The hosted demo is a recorded real run. Live training and uploads need the Docker stack.
- Survival analysis for censored data isn't implemented; run-to-failure logs use classification or RUL regression.
- C-MAPSS FD001 is simulated data with one operating condition. The next step is FD002 and FD004 (multiple operating conditions), then real plant data.
- The drift injection is simulated. A production deployment would feed the monitor from live sensor streams.
- Multiclass targets are rejected with a clear message instead of being guessed at.

13.1 Architecture decision records

- **ADR-001: fixed rules instead of a generative orchestrator.**
- *Context.* A plant reviewer has to be able to reproduce an alert and trace it to the rule that fired.
- *Decision.* No generative model is called anywhere in the runtime decision path. Framing, feature selection and retraining decisions are fixed rules and statistical tests.
- **ADR-002: group the cross-validation folds by unit.**
- *Context.* Sensor readings from a run-to-failure engine are strongly correlated from one cycle to the next.

- *Decision.* Folds are grouped by machine unit (`GroupKFold(5)`), so no unit ever appears in both training and validation.
- **ADR-003: measure PSI on the early-life window.**
- *Context.* A degrading machine's sensors drift as it wears. That's expected wear, not a miscalibrated sensor.
- *Decision.* Baseline distributions are locked to the first 50 healthy cycles, so the monitor doesn't read normal wear as a calibration failure.
- **ADR-004: use native Tree-SHAP in the served model.**
- *Context.* Shipping the full explainer library in every model container adds dependencies.
- *Decision.* Attributions come from the tree library's own contribution output, checked against `shap.TreeExplainer` (maximum difference 0.0), so the container needs nothing extra.

14. Why downtime matters

Siemens' report *The True Cost of Downtime 2024* estimates that the Fortune Global 500 lose about \$1.4 trillion a year to unplanned downtime, roughly 11% of their revenue. I don't make a savings or ROI claim for Sentinel. I evaluated it on public benchmark data, not at a plant, and a real business case would need that plant's own failure and cost history.

15. Data sources, licenses and citations

1. **NASA C-MAPSS FD001 turbofan degradation.** - Source: NASA Prognostics Center of Excellence (PCoE), Ames Research Center (public domain). - Citation: Saxena, A., Goebel, K., Simon, D., & Eklund, N. (2008). *Damage propagation modeling for aircraft engine run-to-failure simulation*. In Proceedings of the 1st International Conference on Prognostics and Health Management (PHM08), Denver, CO.
2. **UCI AI4I 2020 predictive maintenance dataset.** - Source: UCI Machine Learning Repository, dataset #601 (CC BY 4.0). - Citation: Matzka, S. (2020). *Explainable Artificial Intelligence for Predictive Maintenance Applications*. In 2020 Third International Conference on Artificial Intelligence for Industries (AI4I), pp. 69-74. IEEE.
3. **The True Cost of Downtime.** - Citation: Siemens (2024). *The True Cost of Downtime 2024*. The figure is quoted as Siemens reports it; I haven't verified it independently.
4. **Software and fonts.** - Sentinel's code is released under the MIT License. The IBM Plex typefaces are licensed under the SIL Open Font License v1.1.